

Test Script

A step-by-step walkthrough for verifying every part of the site by hand

www.pedmas.com · 2026-08-22

Work through the sections in order — later sections assume the accounts created in earlier ones. Each step says what to do and what you should see. Anything that does not match is a bug: note the step number and what happened instead (the Errors panel at /admin may already have the details).

Where to test: use <http://localhost:3080> to test safely against local data, or <https://www.pedmas.com> to test the real thing. The steps are identical. On the live site, remember Stripe is in sandbox — no real card is ever charged.

A. The public site (no login)

- 1 Open the homepage.**
Expect: Under-construction banner at the very top; "Sign up" absent while registration is closed; hero shows "See a Real Lesson".
- 2 Click "See a Real Lesson" (or the Sample Lesson link in the nav).**
Expect: The column-addition lesson opens with no login. Seven collapsible steps, only the first open.
- 3 Tap through all seven steps, reading as a parent would.**
Expect: Each step opens only after the previous one's button. Step 4 shows the classic mistake ($27 + 15 = 312$) and disproves it with the estimate.
- 4 In step 7, type 62 as the answer.**
Expect: "Correct! Great work" appears, then a Next button; pressing it reveals the closing panel explaining how every lesson is built, with a curriculum link.
- 5 Try /signup directly while registration is closed.**
Expect: A "Not open for new accounts yet" message. No form that actually creates an account.
- 6 Open /curriculum, /pricing, /privacy, /terms.**
Expect: All load. Privacy and Terms carry the "draft pending legal review" notice. Pricing shows \$11.99 + \$5.99, 7-day trial, max 4 children.

B. Admin console

- 7 Log in at /login with the admin credentials (locally: admin@pedmas.com / pedmas-admin; live: the values from Vercel env).**
Expect: Login succeeds and /admin loads with the ADMIN badge.

- 8 Check the overview cards.**
Expect: Counts are plausible: 12 grades, 8 strands, 634 skills; account/student counts match reality.
- 9 In the question preview, pick Grade 1 counting at stage 1, then a Grade 11 topic at stage 5.**
Expect: Both generate. Grade 1 questions read like Grade 1; health verdict is “healthy” for common skills.
- 10 Press “Fire a test error” in the Errors panel.**
Expect: After the reload, a server-source group appears: “Test error from the admin console...” with path POST /api/admin/errors.
- 11 Press Dismiss on that group.**
Expect: It disappears and the panel returns to “No errors recorded”.

B2. Two-factor authentication (admin)

Do this on LOCAL first. Enrolling on production without saving the recovery codes can lock you out of your own admin console.

- 12 In the Two-factor authentication panel, press Turn on.**
Expect: A setup link and a typed key appear, plus a box for a six-digit code. Nothing has changed yet.
- 13 Add it to an authenticator app (paste the otpauth link, or type the key), then enter a WRONG code such as 000000.**
Expect: Rejected with “That code did not match”. Two-factor is still off — a fumbled setup costs nothing.
- 14 Now enter the real six-digit code from the app.**
Expect: Two-factor turns on and TEN RECOVERY CODES appear. Copy them somewhere safe — this is the only time they are shown.
- 15 Press “I have saved them”, then log out and log back in with just the password.**
Expect: You are NOT signed in. Instead an “Enter your code” screen appears. This is the whole point: the password alone is now useless.
- 16 Enter the current code from your app.**
Expect: You reach /admin as normal, and the panel shows two-factor is on with 10 recovery codes left.
- 17 Log out, log in with the password again, and this time enter one of your RECOVERY codes.**
Expect: It works, and you are told how many remain (9). Signing out and trying that SAME recovery code again is rejected — each is single-use.
- 18 In the panel, type a current code and press “Turn off”, then try turning it off again with a wrong code.**
Expect: The first works; a wrong code is refused. Turning it off always needs a current code, so a stolen session cannot remove your protection.

When you are satisfied, repeat the enrolment on PRODUCTION and store the recovery codes with your other critical credentials. Until you do, the admin password is the only thing standing between an attacker and every family's data.

C. A family's journey (the core test)

This is the test that matters most — you are playing a parent and then their child.

- 19 Enable registration first if testing this section (locally: put `REGISTRATION_OPEN=true` in `.env.local` and restart; live: set it in Vercel). Register a fresh parent account.**
Expect: Signup requires accepting the Terms checkbox and the parent/guardian affirmation before it lets you through.
- 20 Add a child: name, grade 3, age 8.**
Expect: Child appears on the parent dashboard with a prompt to start placement.
- 21 Start the placement as the child.**
Expect: It announces it is a placement test before the first question. Questions start EASY (basic addition) and only climb while answers are right.
- 22 Answer the first ~5 questions wrong on purpose.**
Expect: Questions get easier, not harder, and placement ends early with a gentle message — a struggling child is not dragged through fractions.
- 23 Finish placement honestly for a typical 8-year-old.**
Expect: A per-strand report appears (different levels per strand is normal and said to be normal).
- 24 Start today's practice.**
Expect: About 12 questions, one at a time, with a number pad. The current skill's lesson is offered first ("Learn") before practising it.
- 25 Open the lesson, finish it, return to practice.**
Expect: Lesson matches the skill being practised. Back in practice, the same concept appears in the questions.
- 26 Answer one question wrong on purpose.**
Expect: Coaching appears explaining the mistake; you retry until right; a "Show me" worked example is available.
- 27 Complete the whole session.**
Expect: A results screen with first-try accuracy; the streak ticks up; the dashboard reflects the session.
- 28 As the parent, open the child's dashboard page.**
Expect: Progress, streak, mastered skills and time-on-task all visible and matching what the child just did.

D. Billing (sandbox)

- 29 From the parent account, start the subscription with card 4242 4242 4242 4242, any future expiry, any CVC.
Expect: Stripe Checkout shows \$11.99/month for one child and a 7-day trial. After success you land back in the app with the subscription active (trialing).
- 30 Add a second child, then check the Billing page.
Expect: Seat count and monthly amount update to \$17.98 from the next period.
- 31 Open the Stripe customer portal from the Billing page and cancel.
Expect: Cancellation is self-serve; the app shows access continuing to period end.

E. Deletion (the promises the privacy policy makes)

- 32 Re-subscribe (sandbox), then from the Account page delete ONE child profile.
Expect: A confirmation naming the child; after deleting, the profile and its history are gone and billing seats adjust.
- 33 Now delete the ENTIRE account while the subscription is active.
Expect: Explicit confirmation, then logout. In the Stripe sandbox dashboard, the subscription shows CANCELED — an orphaned account must never keep charging.
- 34 Try logging in as the deleted parent.
Expect: "Email or password is incorrect" — the account genuinely no longer exists.

F. Regional behaviour

- 35 With a US connection (or a US VPN) register/log a fresh visit; as any US-region student, practise measurement or geometry.
Expect: Units are inches/feet/miles/°F and spelling is American (meters, color). No metric leaks.
- 36 Repeat from a non-US connection (or an INTL-region account).
Expect: Units are cm/m/km/°C and Commonwealth spelling. No imperial leaks.

G. After email is configured (Resend key set)

- 37 Use "Forgot password" on a real account you control.
Expect: The reset email arrives; the link works once and expires; you can log in with the new password.
- 38 Trigger the weekly summary manually: send GET to /api/cron/weekly-progress with header Authorization: Bearer <CRON_SECRET>.
Expect: Response reports sent/skipped counts; the parent inbox receives the summary; its unsubscribe link works with one click.

39 Fire a test error from /admin.

Expect: Within a minute the admin inbox gets the error alert; firing another immediately does NOT send a second email (6-hour cooldown).

G2. Data retention (dormant accounts)

Accounts nobody has used for 730 days are warned by email and deleted 30 days later. You cannot wait two years to test it, so check the machinery instead:

40 Send GET to /api/cron/retention with NO authorization header.

Expect: 401 Unauthorized. The job that deletes data must never be triggerable by a stranger.

41 Send it again with header Authorization: Bearer <CRON_SECRET>.

Expect: 200, with a JSON report: how many accounts were examined, warned and purged. On a young install: examined = your account count, warned 0, purged 0.

42 Read the “note” field in that reply.

Expect: While RESEND_API_KEY is unset it says so explicitly — no warnings can be sent, so nothing is ever purged. That is the safe direction, and it is stated rather than silent.

43 Open /privacy and read the “How long we keep it” section.

Expect: It states the same 24-month window and 30-day warning as the code, because the page is generated from the same constants.

H. Every skill and every step — how full coverage works

No human can hand-test 634 skills x 5 stages x 2 regions (6,340 combinations) or all 132 lessons' steps — so the machine does, on every single test run:

- Every skill, every stage, both regions: the suite generates and validates questions for all 6,340 combinations (three random seeds each, about 19,000 questions per run). A failure names the exact skill, stage and region.
- Every step of every lesson: an automated walker opens all 132 lessons and taps through every step, failing on anything that crashes, prints a template placeholder, or renders NaN.
- Every skill's five stage names, every skill-to-lesson route, and every advertised error-analysis question are checked exhaustively the same way.
- Higher mathematics gets a SECOND opinion: an independent computer-algebra engine (mathjs, sharing no code with the question generators) re-derives grade 8–12 answers from exactly the text a student sees — solving equations by substitution, comparing algebraic forms at sample points, taking limits and derivatives numerically, integrating, and checking trig values. About 4,500 of the 8,000 senior questions per sweep are verifiable this way; the rest are conceptual or vocabulary questions with nothing to compute. This auditor caught a real bug on its first day (a derivative displayed $23x^2$ where the answer was $6x^2$), and the test suite feeds it planted wrong answers that it MUST catch — so it can never quietly become a rubber stamp.

What the machine cannot judge is whether a question FEELS right for the grade and whether the teaching lands. That is the human job, and it has its own document: the Skill Coverage Checklist PDF lists all 634 skills with a tick-box per stage. Work through it with the admin question preview (/admin) at whatever pace you like — it is designed to be done over days, not in one sitting.

Running the CAS audit yourself

```
cd C:\Users\buruf\Documents\Pedmas
npm run audit:cas          # one-line verdict: checked / passed / failed, plus per-
                           # checker counts
npm run audit:cas:report   # per-family table: grade span, questions, checked,
                           # passed, coverage
```

Expected result: FAILED 0. Any failure prints the exact skill, stage, prompt and answer — that is either a real content bug (fix the generator) or a question whose wording changed (update the checker). Run it after any change to the question generators for grades 8–12.

I. Automated checks (run before every deploy)

```
cd C:\Users\buruf\Documents\Pedmas
npm test          # 284 tests: every skill x stage x region, every lesson
step, CAS sweep
npm run audit:cas # full CAS second opinion on grades 8-12 (expect FAILED 0)
npm run build     # must compile clean – stop the dev server first
```

After a production deploy, three quick probes from any terminal:

```
curl -s -o NUL -w "%{http_code}" https://www.pedmas.com/          (expect 200)
curl -s -X POST https://www.pedmas.com/api/auth/login -H "Content-Type:
application/json" -d '{"email\":"nobody@example.com\","password\":"
"x\"}' (expect a clean 'incorrect' message, not a 500)
curl -s https://www.pedmas.com/sample-lesson | findstr "spill"    (expect the
lesson title)
```

A full pass of sections A–F takes roughly 45 minutes. Sections A, B and the automated checks alone take 10 and are the minimum before any deploy you care about.